



Principais passos:

- Configurar o clock.
- Selecionar os pinos e a função deles: Saída Digital, Entrada Digital, etc.
- Escrever o código para controlar o estado dos pinos e/ou coletar o estado deles.



Categories A-Z

System Core

DMA

✓ GPIO

IWDG

✓ NVIC

✓ RCC

✓ SYS

WWDG

Analog >

Timers >

Connectivity >

Computing >

Middleware a... >

RCC Mode and Configuration

Mode

High Speed Clock (HSE) Crystal/Ceramic Resonator

Low Speed Clock (LSE) Disable

☐ Master Clock Output

Configuration

Reset Configuration

✓ Parameter Settings ✓ User Constants ✓ NVIC Settings ✓ GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

System Parameters

VDD voltage (V) 3.3 V

Prefetch Buffer Enabled

Flash Latency(WS) 2 WS (3 CPU cycle)

RCC Parameters

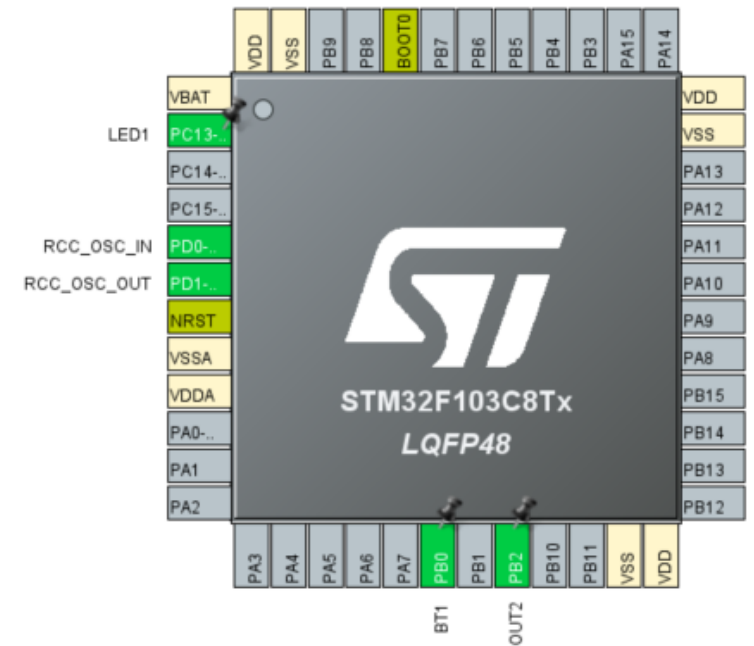
HSI Calibration Value 16

HSE Startup Timeout Value (ms) 100

LSE Startup Timeout Value (ms) 5000

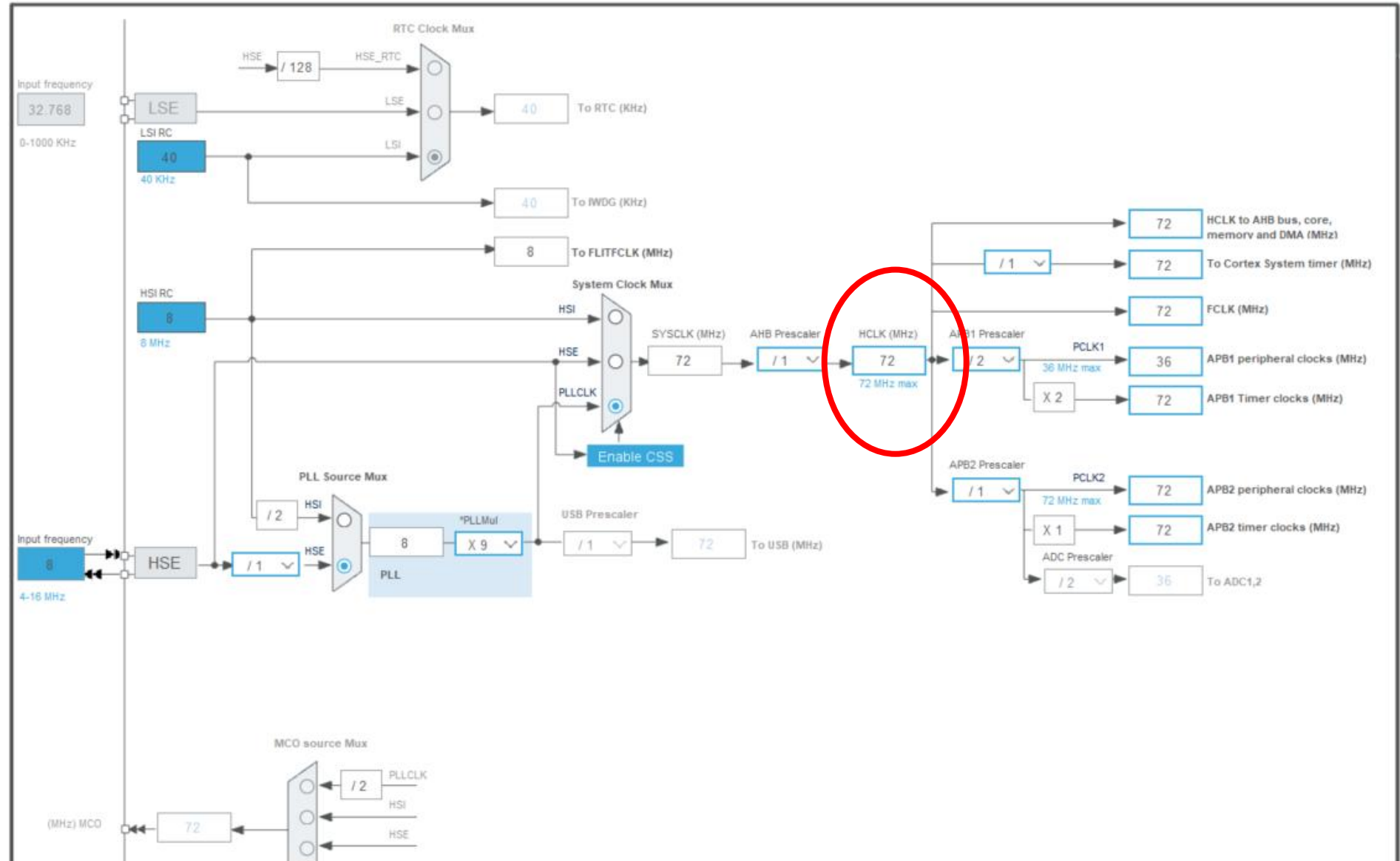
Pinout view

System view





Resolve Clock Issues



Project Run Window Help

Open Project
Close Project

Build All Ctrl+B
Build Configurations
Build Project
Build Working Set
Clean...
Build Automatically

C/C++ Index
Generate Report
Generate Code Alt+K
Properties

Configuration

Clock Configuration

Project Manager

Software Packs Pinout

RCC Mode and Configuration

Mode

High Speed Clock (HSE) Crystal/Ceramic Resonator

Low Speed Clock (LSE) Disable

Master Clock Output

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

System Parameters

VDD voltage (V) 3.3 V

Prefetch Buffer Enabled

Flash Latency (WS) 2 WS (3 CPU cycle)

RCC Parameters

HSI Calibration Value 16

HSE Startup Timeout Value (ms) 100

LSE Startup Timeout Value (ms) 5000

GPIO IWDG
NVIC
RCC
SYS WWDG

Analog
Timers
Connectivity
Computing
Middleware a...

Pinout view System view

STM32F103C8Tx LQFP48

PA3 PA4 PA5 PA6 PA7 PA8 PA9 PA10 PA11 PA12 PA13 PA14 PA15 PA16

PB0 PB1 PB2 PB3 PB4 PB5 PB6 PB7 PB8 PB9 PB10 PB11 PB12 PB13 PB14 PB15

VDD VSS

LED1

RCC_OSC_IN

RCC_OSC_OUT

BT1

OUT2

Após configurar o clock e os pinos que vai usar, use a opção para gerar o código HAL com as configurações.

Write Pin

```
95 while (1)
96 {
97     /* USER CODE END WHILE */
98
99     HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
100    HAL_Delay(200);
101    HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
102    HAL_Delay(200);
```

Toggle Pin

```
95 while (1)
96 {
97     /* USER CODE END WHILE */
98
99     HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
100    HAL_Delay(500); // Espera 500ms
101
```

HAL_GPIO_WritePin

Esta função é usada para definir o estado de um pino GPIO específico, podendo colocá-lo em nível alto (HIGH/SET) ou baixo (LOW/RESET).

void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState);

Parâmetros:

- **GPIOx:** Ponteiro para o registro da porta GPIO (GPIOA, GPIOB, GPIOC, etc.)
- **GPIO_Pin:** Especifica o pino a ser configurado, usando macros como GPIO_PIN_0, GPIO_PIN_1, ..., GPIO_PIN_15
- **PinState:** Estado desejado para o pino, podendo ser:
 - GPIO_PIN_SET (ou 1): Coloca o pino em nível alto (3.3V)
 - GPIO_PIN_RESET (ou 0): Coloca o pino em nível baixo (0V)

HAL_GPIO_TogglePin

Esta função inverte o estado atual de um pino GPIO específico. Se o pino estiver em nível alto, a função o coloca em nível baixo, e vice-versa.

```
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
```

Parâmetros:

- **GPIOx**: Ponteiro para o registro da porta GPIO (GPIOA, GPIOB, GPIOC, etc.)
- **GPIO_Pin**: Especifica o pino a ser invertido, usando macros como GPIO_PIN_0, GPIO_PIN_1, ..., GPIO_PIN_15

Funcionamento interno:

- A função HAL_GPIO_TogglePin lê o estado atual do pino usando o registro ODR (Output Data Register), inverte o bit usando uma operação XOR e escreve o novo valor de volta no registro ODR.

Categories A->Z

System Core

DMA

☒ GPIO

IWDG

☒ NVIC

☒ RCC

☒ SYS

WWDG

Analog

Timers

Connectivity

Computing

Middleware a...

GPIO Mode and Configuration

Mode

Configuration

Group By Peripherals

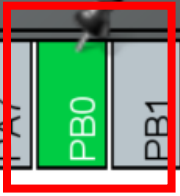
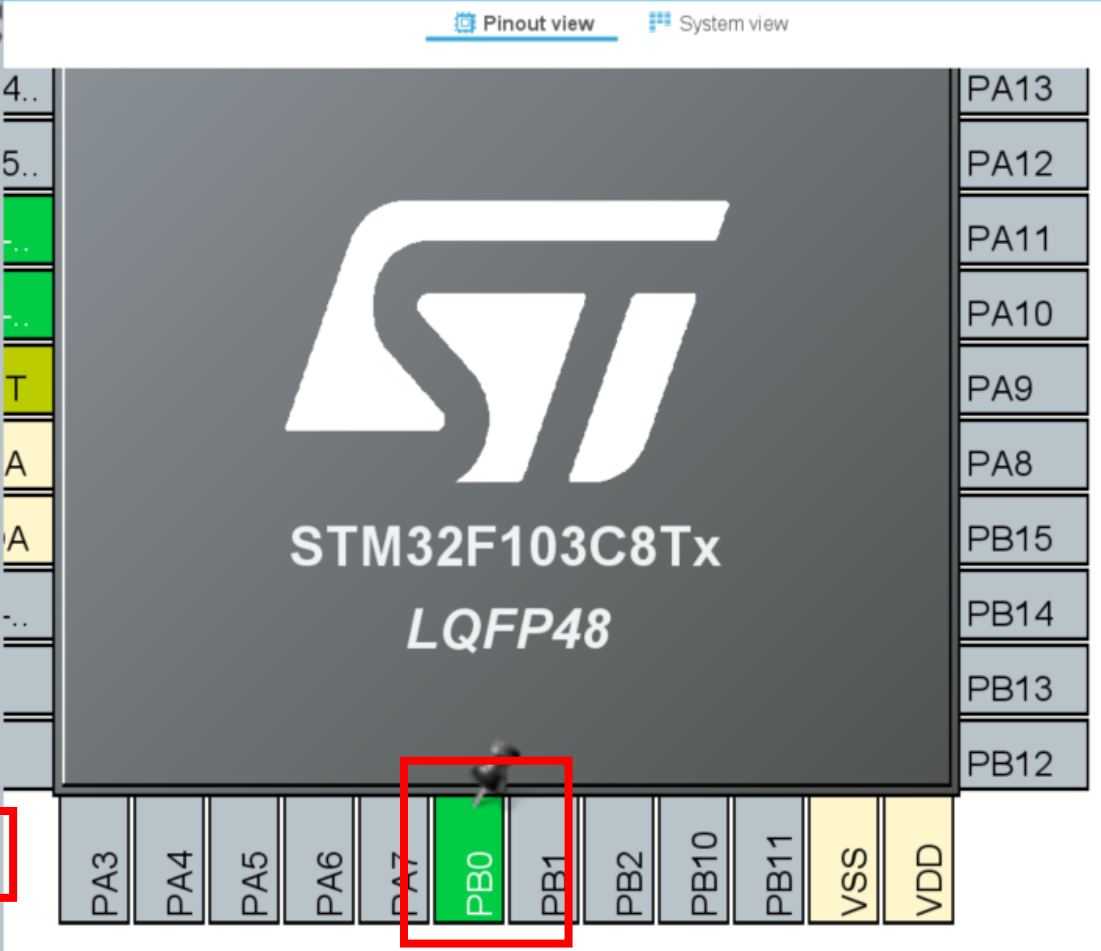
☒ GPIO ☒ RCC

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin Name	Signal on Pin	GPIO output le...	GPIO mode	GPIO Pull-up...	Maximum out...	User Label	Modified
PB0	n/a	n/a	Input mode	Pull-up	n/a	BT1	☒
PC13-IAMP...	n/a	Low	Output Push Pull	No pull-up an...	Low	LED1	☒



HAL_GPIO_ReadPin

Função para Leitura de Entrada Digital. A função HAL_GPIO_ReadPin é utilizada na biblioteca HAL (Hardware Abstraction Layer) do STM32 para realizar a leitura do estado atual de um pino GPIO configurado como entrada digital.

GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);

Parâmetros

- **GPIOx:** Ponteiro para o registro da porta GPIO (GPIOA, GPIOB, GPIOC, etc.)
- **GPIO_Pin:** Especifica o pino a ser lido, usando macros como GPIO_PIN_0, GPIO_PIN_1, ..., GPIO_PIN_15

```
95 while (1)
96 {
97     // Verifica se um botão conectado ao PB0 está pressionado
98     // Considerando um botão com resistor de pull-up (lógica invertida)
99     if(HAL_GPIO_ReadPin(BT1_GPIO_Port, BT1_Pin) == GPIO_PIN_RESET)
100     {
101         // Botão pressionado (conectado ao GND)
102         HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
103     }
104     else
105     {
106         // Botão não pressionado
107         HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
108     }
109
110 }
111 /* USER CODE END 3 */
112 }
```

Debounce de Botão

O ***debounce*** é uma técnica essencial quando trabalhamos com botões físicos em microcontroladores. Quando um botão mecânico é pressionado ou liberado, os contatos metálicos não fazem uma transição limpa entre os estados, mas sim "quicam" (bounce) várias vezes por um curto período (geralmente 5-20ms), gerando múltiplos pulsos falsos que podem ser interpretados como vários pressionamentos.

Botão - "Bounce"



Por que o Debounce é Necessário?

Sem debounce, um único pressionamento de botão pode resultar em:

- Múltiplas ações sendo executadas
- Contagens incorretas
- Comportamento imprevisível no sistema

Debounce com Delay Simples

Esta é a técnica mais básica e consiste em esperar um período curto após detectar uma mudança no estado do botão.

```
191
192 GPIO_PinState verificarBotao(void) {
193     // Lê o estado atual do botão
194     GPIO_PinState estadoAtualBotao = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_0);
195     HAL_Delay(50); // Delay 50 ms
196     return(estadoAtualBotao);
197 }
```

Debounce verificando estado anterior com interlock

Como Funciona:

- Mantemos dois estados: a última leitura (ultimoEstadoBotao) e o estado confirmado estável (estadoEstavel);
- Quando detectamos uma mudança na leitura, reiniciamos o temporizador;
- Só atualizamos o estado estável após o período de debounce;
- Usamos uma flag adicional (acaoBotaoExecutada) para garantir que a ação seja executada apenas uma vez por pressionamento.

```

/* Private typedef -----
typedef struct {
    GPIO_TypeDef *port;
    uint16_t pin;
    uint32_t debounce_time;    // tempo de debounce em ms
    uint32_t last_change_time;  // último timestamp de mudança
    uint8_t ultimoEstadoEstavel; // último estado estável
    uint8_t raw_state;          // leitura crua
    uint8_t pressionado;
    uint8_t solto;
    uint32_t press_time;        // quando foi pressionado
} Button_t;

/* Private define -----
#define PI 3.14;

/* Private variables -----
//Definição das variáveis globais
UART_HandleTypeDef huart1;
char msg[50]; //Vetor para armazenar a string enviada na UART
Button_t botao1;

```

```

//*****
void Button_Init(Button_t *btn, GPIO_TypeDef *port, uint16_t pin, uint32_t
    btn->port = port;
    btn->pin = pin;
    btn->debounce_time = debounce_ms;
    btn->ultimoEstadoEstavel = HAL_GPIO_ReadPin(port, pin);
    btn->raw_state = btn->ultimoEstadoEstavel;
    btn->last_change_time = HAL_GetTick();
    btn->pressionado = 0;
    btn->solto = 0;
    btn->press_time = 0;
}

```

```

//*****
void lerBotao(Button_t *btn) {
    uint8_t estadoAtual = HAL_GPIO_ReadPin(btn->port, btn->pin);
    uint32_t now = HAL_GetTick();

    // Reset dos eventos
    btn->pressionado = 0;
    btn->solto = 0;

    if (estadoAtual != btn->raw_state) {
        // mudança detectada, reinicia timer
        btn->raw_state = estadoAtual;
        btn->last_change_time = now;
    }

    // Confirma mudança apenas se passou o tempo de debounce
    if ((now - btn->last_change_time) >= btn->debounce_time)
    {
        if (btn->ultimoEstadoEstavel != btn->raw_state)
        {
            btn->ultimoEstadoEstavel = btn->raw_state;

            if (btn->ultimoEstadoEstavel == GPIO_PIN_RESET)
            { // Botão pressionado (pull-up típico)
                btn->pressionado = 1;
                btn->press_time = now;
            }
            else
            {
                btn->solto = 1;
            }
        }
    }
}
//*****

```

Função HAL_GetTick

A função HAL_GetTick() é uma das funções mais úteis e frequentemente utilizadas na biblioteca HAL (Hardware Abstraction Layer) dos microcontroladores STM32. Ela fornece um contador de tempo para várias aplicações, como timeouts, delays não-bloqueantes e **debounce de botões**.

uint32_t HAL_GetTick(void);

Internamente, HAL_GetTick() utiliza um temporizador do sistema chamado SysTick, que é um componente padrão dos microcontroladores ARM Cortex-M. O SysTick é um contador decrescente que gera uma interrupção em intervalos regulares, tipicamente configurado para 1 milissegundo.

A função HAL_GetTick() simplesmente retorna o valor atual deste contador